

Lab Exercise — Module 3

Overview of Honeypots: Connect, Attack & Analyze

Duration: 90 minutes | Instructor: Abhishek | Day 2

The Scenario:

Your SOC manager has asked you to connect a running honeypot to the team's Wazuh SIEM. Once confirmed live, you will simulate four common attacks against it, watch the alerts appear in the Dashboard, and write a short analysis of what the "attacker" did. By the end of this lab, you will understand exactly what an attacker sees — and exactly what you see on the other side.

Safety Notice: All attacks in this lab must be directed **only** at your assigned honeypot VM 10.30.5.40. Never target any other IP on this network.

What You Will Accomplish

By the end of this lab, you will have:

- Verified that Cowrie, Dionaea, Suricata, and H0neytr4p containers are running
- Confirmed all T-Pot log paths are connected to the Wazuh agent
- Tested decoders and rules using `wazuh-logtest`
- Simulated four real attacks and found each as an alert in the Dashboard
- Analyzed attack patterns using WQL queries
- Written an attacker profile based on captured honeypot data

Prerequisites

- Honeypot VM 10.30.5.40 already running T-Pot (instructor pre-deployed)
- Wazuh agent installed on honeypot VM and showing **Active** in the Dashboard
- Attacker VM 10.30.5.20 provided by instructor

Lab Environment

Honeypot VM IP:	10.30.5.40
Honeypot SSH:	<code>ssh -i HoneyPotController.pem -p 4567 admin@10.30.5.40</code>
Attacker VM IP:	10.30.5.20
Wazuh Dashboard URL:	<code>https://10.30.5.10:443</code>
T-Pot data root:	<code>/home/admin/tpotce/data/</code>
Wazuh agent config:	<code>/var/ossec/etc/ossec.conf</code>
Wazuh agent log:	<code>/var/ossec/logs/ossec.log</code>

Task 1 — Verify T-Pot is Running (10 min)

T-Pot is pre-deployed on your honeypot VM. Confirm all services are healthy before running any attacks.

Step 1.1 — Check critical containers are Up

```
sudo docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" \
| grep -E "cowrie|dionaea|suricata|h0neytr4p"
```

Checkpoint: All four services must show Up.

Record:

Cowrie status: _____ Dionaea status: _____

Suricata status: _____ H0neytr4p status: _____

Step 1.2 — Confirm log files exist

```
for f in \
/home/admin/tpotce/data/cowrie/log/cowrie.json \
/home/admin/tpotce/data/dionaea/log/dionaea.json \
/home/admin/tpotce/data/suricata/log/eve.json \
/home/admin/tpotce/data/h0neytr4p/log/log.json; do
    [ -f "$f" ] && echo "OK $f" || echo "MISSING: $f"
done
```

All four must show OK. If any show MISSING — the container has not received traffic yet. Run `nc -z -w2 10.30.5.40 22` from the attacker VM to wake Cowrie, then re-check.

Step 1.3 — Confirm Wazuh agent is connected

```
sudo systemctl status wazuh-agent | grep Active
sudo tail -3 /var/ossec/logs/ossec.log
```

Checkpoint: Must show active (running) and no Unable to connect errors. If disconnected — raise your hand. Do not proceed until the agent shows Connected.

Task 2 — Explore the Raw Honeypot Logs (10 min)

Before looking at Wazuh, read the logs directly to understand what each honeypot records.

Step 2.1 — Read Cowrie logs

```
sudo tail -5 /home/admin/tpotce/data/cowrie/log/cowrie.json \
| python3 -c "
import sys, json
for line in sys.stdin:
    line = line.strip()
```

```
    if line:
        print(json.dumps(json.loads(line), indent=2))
        print('---')
"
```

Note the fields you see (fill in from actual output):

- eventid: _____ (e.g., cowrie.login.failed)
- src_ip: _____
- username: _____
- password: _____

Step 2.2 — Read Dionaea logs

```
sudo tail -5 /home/admin/tpotce/data/dionaea/log/dionaea.json \
| python3 -c "
import sys, json
for line in sys.stdin:
    line = line.strip()
    if line:
        print(json.dumps(json.loads(line), indent=2))
        print('---')
"
```

Look for the `connection.type` and `connection.protocol` fields.

Record: Which protocol appeared most in Dionaea logs?

Step 2.3 — Read H0neytr4p logs

```
sudo tail -5 /home/admin/tpotce/data/h0neytr4p/log/log.json \
| python3 -c "
import sys, json
for line in sys.stdin:
    line = line.strip()
    if line:
        print(json.dumps(json.loads(line), indent=2))
        print('---')
"
```

Look at the `trapped` field. When is it true vs false?

Record: What does `trapped: true` mean for H0neytr4p?

Step 2.4 — Check Suricata honeypot alerts

```
sudo python3 -c "
import json
with open('/home/admin/tpotce/data/suricata/log/eve.json') as f:
    for line in f:
        try:
```

```
e = json.loads(line)
if e.get('event_type') == 'alert':
    print(e.get('alert', {}).get('signature', ''))
except: pass
" | sort | uniq -c | sort -rn | head -10
```

Key log paths confirmed in this task:

```
/home/admin/tpotce/data/cowrie/log/cowrie.json
/home/admin/tpotce/data/dionaea/log/dionaea.json
/home/admin/tpotce/data/suricata/log/eve.json
/home/admin/tpotce/data/h0neytr4p/log/log.json
```

Task 3 — Test Decoders and Rules with wazuh-logtest (10 min)

Confirm Wazuh can decode honeypot logs and fire the right rules — before running any real attacks. This task is demonstrated live by the instructor. Follow along and record the results.

Step 3.1 — Test Cowrie failed login

Paste at the logtest prompt:

```
{"eventid": "cowrie.login.failed", "src_ip": "99.99.99.99",
"username": "admin", "password": "test123",
"timestamp": "2026-06-11T10:00:00Z"}
```

Expected: Phase 2 extracts eventid, src_ip, username, password. Phase 3 fires Rule 101101, level 7.

Record:

Rule ID fired: _____ Level: _____ Description: _____

Step 3.2 — Test H0neytr4p trapped request

```
{"dest_port": "443", "request_method": "GET",
"request_uri": "/.env", "src_ip": "99.99.99.99",
"trapped": "true", "trapped_for": "env_config_file",
"timestamp": "2026-06-11T10:00:00Z"}
```

Expected: Rule 101301, level 12 — *H0neytr4p: TRAPPED attacker*

Record: Rule ID: _____ Level: _____ Description: _____

Step 3.3 — Test Dionaea SMB

```
{"src_ip": "99.99.99.99", "dst_port": "445",
"connection.type": "accept", "connection.protocol": "smbd",
"timestamp": "2026-06-11T10:00:00Z"}
```

Expected: Rule 101201, level 10 — *Dionaea: SMB connection — possible EternalBlue*

Record: Rule ID: _____ Level: _____ Description: _____

Task 4 — Attack 1: Nmap Port Scan (10 min)

An attacker's first step is always reconnaissance. They want to know what ports are open.

Step 4.1 — Run Nmap from the attacker VM

```
# Run from the ATTACKER VM (10.30.5.20)
sudo nmap -sS -p 21,22,23,80,443,445,1433,3306,5900,6379,8080 10.30.5.40
```

Step 4.2 — Find the alert in Wazuh Dashboard

1. Go to **Security Events** in the Dashboard.
2. Filter: Agent = **Honeypot**.
3. Search: rule.groups:honeybot
4. Look for a Suricata alert mentioning “Nmap” or “port scan”.

Record:

Rule description: _____

Rule level: _____ Agent name: _____

Which honeypot service logged the most connection attempts? _____

Task 5 — Attack 2: SSH Brute Force on Cowrie (15 min)

After port scanning, attackers try to break into SSH. Cowrie captures every single attempt.

Step 5.1 — Create a wordlist on the attacker VM

```
cat > /tmp/passwords.txt << 'EOF'
123456
password
admin
root
letmein
qwerty
toor
password123
test
welcome
EOF
```

Step 5.2 — Run Hydra from the attacker VM

```
# Run from the ATTACKER VM (10.30.5.20)
hydra -l root -P /tmp/passwords.txt -t 4 ssh://10.30.5.40
```

Step 5.3 — Watch Cowrie collect the credentials live

While Hydra is running, on the honeypot VM:

```
sudo tail -f /home/admin/tpotce/data/cowrie/log/cowrie.json \
| python3 -c "
import sys, json
for line in sys.stdin:
    try:
        e = json.loads(line.strip())
        if 'login' in e.get('eventid','') or 'command' in e.get('eventid',
        '',''):
            print(e['eventid'], '| user=', e.get('username',''),
                'pass=', e.get('password',''),
                'cmd=', e.get('input',''),
                'from=', e.get('src_ip',''))
    except: pass
"
```

Step 5.4 — Find the brute-force alerts in the Dashboard

Search in Security Events:

```
rule.id:101101 AND agent.name:Honeypot
```

Record from Dashboard:

How many Cowrie login failed alerts (101101) appeared? _____

Did the brute-force correlation rule fire (101105, level 14)? Yes / No: _____

Did a login SUCCESS alert fire (101102, level 12)? Yes / No: _____

Did a command execution alert fire (101103, level 10)? Yes / No: _____

List 3 username/password combinations Cowrie captured:

1. _____ 2. _____ 3. _____

Task 6 — Attack 3: SMB Probe on Dionaea (10 min)

Dionaea on port 445 catches Windows exploit attempts. We simulate an EternalBlue-style scan.

Step 6.1 — Run an SMB scan from the attacker VM

```
# Run from the ATTACKER VM (10.30.5.20)
nmap -sV -p 445 --script smb-vuln-ms17-010 10.30.5.40
```

Step 6.2 — Find the Dionaea alert in the Dashboard

Search in Security Events:

```
rule.id:101201
```

Record:

Rule description: _____

Rule level: _____ Rule ID: _____

Did a “possible EternalBlue” alert fire? Yes / No: _____

Task 7 — Attack 4: H0neytr4p Web Probe (10 min)

H0neytr4p on port 443 catches web vulnerability scanners. Some paths trigger a trap — meaning the request matched a known attacker pattern.

Step 7.1 — Run web probes from the attacker VM

```
# Basic probes --- rule 101300, level 5
curl -sk https://10.30.5.40/ -o /dev/null
curl -sk https://10.30.5.40/admin -o /dev/null
curl -sk https://10.30.5.40/login -o /dev/null

# Sensitive paths --- rule 101301, level 12 (TRAPPED)
curl -sk https://10.30.5.40/.env -o /dev/null
curl -sk https://10.30.5.40/wp-config.php -o /dev/null
curl -sk https://10.30.5.40/.git/config -o /dev/null
```

Step 7.2 — Watch H0neytr4p log live

On the honeypot VM:

```
sudo tail -f /home/admin/tpotce/data/h0neytr4p/log/log.json \
| python3 -c "
import sys, json
for line in sys.stdin:
    try:
        e = json.loads(line.strip())
        trapped = e.get('trapped','false')
        mark = 'TRAPPED' if trapped == 'true' else 'probe '
        print(mark, '|', e.get('request_method',''),
              e.get('request_uri',''), 'from', e.get('src_ip',''))
    except: pass
"
```

Step 7.3 — Find H0neytr4p alerts in the Dashboard

Search in Security Events:

```
rule.groups:honeytrap
```

Record:

How many probe alerts (101300, level 5) appeared? _____

How many TRAPPED alerts (101301, level 12) appeared? _____

Which URI triggered the trap? _____ Alert level: _____

What does the trapped_for field say? _____

Task 8 — Analyze Attack Patterns with WQL (10 min)

Now you are the analyst. All four attacks came from the same IP. Build a picture of what they did.

Step 8.1 — Find all events from the attacker IP

In Security Events:

```
data.src_ip:10.30.5.20
```

Step 8.2 — Check for multi-service campaign alert

```
rule.id:101900
```

Rule 101900 fires when one IP hits **multiple** honeypot services within 5 minutes. It is the highest-severity honeypot rule (level 15).

Did it fire? Yes / No: _____

If No — the attacks may have been spread too far apart. Run the quick sequence from the cheatsheet and try again.

Step 8.3 — Timeline of all honeypot events from your attacker

```
data.src_ip:10.30.5.20 AND rule.groups:honeybot
```

Sort by timestamp ascending. Write the sequence:

1. _____
2. _____
3. _____
4. _____
5. _____

Task 9 — Write an Attacker Profile (10 min)

Every attacker leaves a fingerprint in the logs. Based on what you captured, write a short profile.

Fill in from your honeypot data:

Attacker IP: _____

First seen: _____

Last seen: _____

Services targeted: _____

Top username tried: _____

Top password tried: _____

Web paths probed: _____

Web paths that trapped: _____

Tools used (if known): _____

Highest alert level: _____

Rule 101900 fired? _____

Reflection Questions

Answer in your own words (3–4 sentences each):

- Q1.** Cowrie captured 50 failed SSH logins from one IP in 2 minutes, all trying different user-name/password combinations. Is this a targeted attack or automated scanning? How can you tell the difference from the log data?
- Q2.** Your honeypot is internet-facing and receives 2,000 hits in 24 hours from 300 different IPs. Your colleague wants to automatically block all those IPs at the perimeter firewall. What is the risk of doing this without manual review? Give one concrete example of what could go wrong.
- Q3.** H0neytr4p fired rule 101301 (TRAPPED, level 12) when an attacker requested `/.env`. Why is a request to `/.env` more suspicious than a request to `/admin`? What would an attacker find in a `.env` file on a real server?

- Q4.** Write the Wazuh rule XML (Rule ID 101106, level 12) that would detect any `cowrie.session.file_download` event — meaning an attacker uploaded or downloaded a file from the fake shell.

Quick Reference: T-Pot Log Paths

Log Path	Service	What it Contains
/home/admin/tpotce/data/cowrie/log/cowrie.json	Cowrie	SSH/Telnet sessions
/home/admin/tpotce/data/dionaea/log/dionaea.json	Dionaea	SMB, FTP, MySQL probes
/home/admin/tpotce/data/suricata/log/eve.json	Suricata	IDS alerts on all traffic
/home/admin/tpotce/data/h0neytr4p/log/log.json	H0neytr4p	HTTPS probe logs
/home/admin/tpotce/data/conpot/log/	Conpot	ICS/SCADA probes
/home/admin/tpotce/data/heralding/log/log_session.json	Herald	Credential attempts
/var/ossec/etc/ossec.conf	Wazuh	Agent configuration
/var/ossec/logs/ossec.log	Wazuh	Agent own log

Quick Reference: Rule IDs

Rule ID	Service	Level	Description
101101	Cowrie	7	SSH login failed
101102	Cowrie	12	SSH login SUCCESS
101103	Cowrie	10	Command executed in fake shell
101104	Cowrie	12	Malware file downloaded
101105	Cowrie	14	SSH brute force (5 failures / 60s)
101201	Dionaea	10	SMB connection — possible EternalBlue
101300	H0neytr4p	5	HTTPS probe (not trapped)
101301	H0neytr4p	12	TRAPPED request (e.g. /.env, /wp-config.php)
86601	Suricata	3	IDS alert on honeypot traffic
101900	Correlation	15	Same IP hits 3+ services in 5 minutes

Submission Checklist

Before you leave, confirm you have:

- ☐ Screenshot of `docker ps` showing `cowrie`, `dionaea`, `suricata`, `h0neytr4p` all **Up**
- ☐ Screenshot of `wazuh-logtest` output showing Cowrie JSON decoded and rule 101101 fired
- ☐ Screenshot of Nmap scan alert in the Wazuh Dashboard
- ☐ Screenshot of SSH brute-force alerts (rule 101101) in the Dashboard
- ☐ Screenshot of Dionaea SMB alert (rule 101201) in the Dashboard
- ☐ Screenshot of H0neytr4p TRAPPED alert (rule 101301) in the Dashboard
- ☐ Completed Attacker Profile table (Task 9)
- ☐ Written answers to all four reflection questions